Automated Trading System

Technical Architecture, Tech Stack, and Local Setup Guide

Generated for the Codex-created application copy in automated_trading_system_codex.

1. What Was Created

A clean and improved Python automated trading application was created for BTC/USDT and XAU/USD. The application includes data feeds, technical indicators, multi-strategy signal generation, risk management, portfolio accounting, simulated execution, backtesting, reporting, and a Flask dashboard for monitoring paper/live trading sessions.

Reliability fixes were included in this copy: corrected short-position accounting, mark-to-market risk value refreshes before trade approval, custom balance handling in backtest metrics, safer default live leverage, and focused portfolio accounting tests.

2. Technical Architecture

- CLI entry point: main.py selects backtest, paper, or live mode and wires the
- system components together.
- Configuration layer: config/settings.py defines data feed, strategy, risk,
- execution, monitoring, currency, and backtest settings.
- Data layer: src/data fetches BTC and gold market data from Binance/CCXT, Yahoo
- Finance, CoinGecko, Alpha Vantage, OANDA, and fallback sources where configured.
- Feature layer: src/indicators calculates technical indicators, volatility, and
- regime features used by the strategies.
- Strategy layer: src/strategies contains trend-following, mean-reversion, breakout,
- momentum, volatility, correlation/macro, grid, ML, and ensemble strategies.
- Risk layer: src/risk enforces drawdown, exposure, allocation, stop-loss, take-
- profit, trailing-stop, and position-sizing controls.
- Execution layer: src/execution models orders, portfolio state, simulated broker
- behavior, and optional live broker integrations.
- Backtesting layer: src/backtesting simulates historical trading, captures
- equity/trade logs, and calculates performance metrics.
- Monitoring layer: src/monitoring provides the Flask dashboard, REST APIs, alerts,
- structured logs, and state recovery helpers.
- Visualization layer: src/visualization generates charts for backtest and trading
- reports.

3. Tech Stack

- Language: Python 3.10+ recommended.
- Data and computation: pandas, numpy, scipy, yfinance.
- Technical analysis: ta.
- Machine learning: scikit-learn, XGBoost, PyTorch, joblib.
- Market/exchange connectivity: requests, websocket-client, aiohttp, ccxt.
- Scheduling: schedule, APScheduler.
- Storage: SQLite through SQLAlchemy.
- Web dashboard/API: Flask, Flask-SocketIO, Flask-CORS, Jinja2.

- Charts/reports: matplotlib, seaborn, TradingView Lightweight Charts in the
- dashboard.
- Logging/terminal output: loguru, rich.
- Testing and quality: pytest, pytest-asyncio, pytest-cov, flake8, black, isort.
- Containerization: Docker and Docker Compose.

4. Local Setup Guide

Use the commands below from Windows PowerShell unless your environment differs.

```
cd D:\Training_Materials\AI\algo_trading\web\live\automated_trading_system_codex
```

Step 1: Create a Python Virtual Environment

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Step 2: Install Dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PyTorch installation is slow or platform-specific, install the correct wheel from the official PyTorch selector for your CPU/GPU setup, then rerun the requirements installation if needed.

Step 3: Configure Environment Variables

```
Copy-Item .env.example .env
```

For backtest and paper mode, API keys are optional because public/free data feeds are used. For live mode, set only the broker credentials you intend to use. Keep BINANCE_TESTNET, DELTA_TESTNET, and XM_DEMO enabled while testing.

Step 4: Run a Backtest

```
python main.py --mode backtest
python main.py --mode backtest --visualize
```

Backtest mode fetches historical BTC/USDT and XAU/USD data, runs the strategy ensemble, prints the performance report, and optionally writes charts under reports/charts.

Step 5: Run Paper Trading Locally

```
python main.py --mode paper --balance 100000 --dashboard-port 5000
```

Open the dashboard at http://localhost:5000. Paper mode uses simulated execution and is the preferred local validation mode before any live broker configuration is enabled.

Step 6: Run Tests

```
pytest tests -q
```

If pytest is not recognized, install dependencies inside the virtual environment first. The Codex copy also includes tests/test_portfolio.py to validate long and short portfolio accounting.

Step 7: Optional Docker Run

```
docker compose --profile backtest up backtester
docker compose up trading-system
```

When using Docker, ensure persistent volumes are used for data, logs, reports, and state if you plan to keep runtime history.

5. Important Reliability Notes

- Start with backtest and paper mode only. Live trading uses real funds and should
- be enabled only after credentials, exchange permissions, and risk limits are
- reviewed.
- Default live leverage in the Codex copy is 1x. Set LEVERAGE explicitly only after
- confirming broker margin behavior and maximum-loss scenarios.
- The Flask dashboard binds to port 5000 by default. For production hosting, place
- it behind a reverse proxy with HTTPS, authentication, and firewall restrictions.
- Before AWS deployment, add a pinned lock file, CI tests, secrets management,
- CloudWatch/log shipping, health checks, backups, and a documented rollback plan.

6. Key Files

- main.py: application entry point and mode selection.
- requirements.txt: Python dependency list.
- config/settings.py: main configuration dataclasses.
- src/execution/portfolio.py: portfolio and PnL accounting.
- src/risk/risk_manager.py: exposure, drawdown, and trade approval logic.
- src/monitoring/dashboard.py: dashboard and REST API.
- CODEX_SUMMARY.md: summary of Codex changes and recommended next improvements.